



OOP_ASSIGNMENT

COURSE CODE: CSP610



TUTOR: DR. A.J. FOFANAH

Students ID: 28328:19742

MSC. COMPUTER SCIENCE

CONTENTS

Declaration	2
1.0 Abstract	3
2.0 Introduction And Design Overview	4
3.0 Solid Analysis And Refactoring (Question 1.1)	5
4.0 Design Pattern Justifications With Uml Diagrams	6
Figure 1: Builder Pattern Uml Diagram.....	7
Figure 2: Decorator Pattern Uml Diagram.....	7
Figure 3: Observer Pattern Uml Diagram	8
5.0 Generics And Type System Discussion (Question 3.2).....	9
6. Stream Api And Functional Programming Reflections	10
7. Concurrency Design Decisions (Question 5.3).....	11
8. Modern Java Features Discussion (Question 6).....	12
9. Test Coverage Summary (Include Jacoco Report Screenshot).....	13
10. Conclusion And Lessons Learned	17
11. References.....	18

DECLARATION

By submitting this assignment, I certify that this work is solely my own. All code is original or properly attributed. I have not shared my solution with any other student. I understand the consequences of academic dishonesty as outlined in the University's student conduct policy.

Students ID: 28328:19742

Date: 12 June,2026.

1.0 ABSTRACT

SmartLib is a Java 17-based library management application created to explore and demonstrate key object-oriented programming concepts and modern software engineering practices. The project was developed in six progressive phases, beginning with domain modelling and advancing through SOLID-based refactoring, design patterns, generics, functional programming, concurrency, and contemporary Java language features. Throughout development, emphasis was placed on producing clean, maintainable, and testable code. The system incorporates immutable data structures, repository abstractions, borrowing policies, event-driven interactions, and asynchronous processing to support core library operations. Automated testing, UML modelling, and JaCoCo coverage analysis were used to validate both functionality and design quality. This report examines the architectural decisions made during each stage of development and evaluates their contribution to system flexibility, scalability, and maintainability. As a result, SmartLib serves not only as a functional software solution but also as a practical demonstration of object-oriented design principles and modern Java development techniques

2.0 INTRODUCTION AND DESIGN OVERVIEW

SmartLib is a library management system developed to demonstrate the practical application of object-oriented programming principles and modern software engineering techniques. The system supports core library functions, including managing books, members, loans, reservations, fines, notifications, analytics, and inventory records. Although the application operates within a relatively small problem domain, it incorporates many of the challenges commonly encountered in larger software systems, including object creation, extensibility, event handling, type safety, concurrency, and modular design.

Rather than being developed as a single large solution, SmartLib was implemented incrementally through a series of structured phases. Each phase introduced a new set of concepts and architectural improvements, allowing functionality to be tested and validated before further enhancements were added. This approach reduced complexity while ensuring that every design decision remained purposeful and traceable.

The architecture is centred around the domain model, with the `smartlib.domain` package serving as the foundation of the application. Additional packages build upon this core to demonstrate design patterns, generics, functional programming techniques, concurrency mechanisms, and modern Java language features. Supporting artefacts, including UML diagrams, automated tests, a JPMS module descriptor, and JaCoCo coverage reports, provide further evidence of the system's design quality and maintainability.

3.0 SOLID ANALYSIS AND REFACTORING (QUESTION 1.1)

A major architectural improvement within SmartLib was the transition from a manager-centred design to a collection of smaller, specialised components, each responsible for a specific aspect of the system. This restructuring was guided by the SOLID principles and resulted in a codebase that is easier to understand, maintain, and extend.

The Single Responsibility Principle (SRP) is demonstrated through the clear separation of concerns across the application. Loan processing, fine calculation, notification handling, reporting, and data persistence are managed by independent components rather than being grouped into a single class. For instance, the `DefaultLoanService` coordinates borrowing and return activities, while fine calculation, notification delivery, and persistence operations are delegated to dedicated services.

The Open/Closed Principle (OCP) is evident in the borrowing policy framework. Different membership categories, including `STANDARD`, `PREMIUM`, and `STUDENT`, are represented by individual implementations of the `BorrowingPolicy` interface. This design allows new borrowing rules to be introduced without altering existing workflow logic.

The Liskov Substitution Principle (LSP) is supported by abstractions such as `Repository<T, ID>` and `BorrowingPolicy`. These abstractions enable different implementations to be substituted without affecting the correctness of the system, provided they adhere to the expected contract.

The Interface Segregation Principle (ISP) is reflected in the use of small, focused interfaces. Components interact only with the functionality they require, preventing unnecessary dependencies and reducing coupling between unrelated features.

Finally, the Dependency Inversion Principle (DIP) is achieved through constructor-based dependency injection and interface-driven design. High-level services depend on abstractions rather than concrete implementations, improving flexibility and simplifying testing. This design decision proved particularly beneficial during the implementation of design patterns, concurrency features, and automated tests.

4.0 DESIGN PATTERN JUSTIFICATIONS WITH UML DIAGRAMS

Three Gang of Four design patterns were implemented in SmartLib because each addressed a specific architectural need within the project. Rather than being added only for demonstration purposes, the patterns were selected to solve practical design problems related to object construction, behavioural extension, and event-driven coordination.

The Builder pattern is implemented through `LoanBuilder`. This pattern was appropriate because a `Loan` contains both mandatory information, such as the member and book, and optional information, such as notes, renewal count, and reference code. Using a builder avoids long telescoping constructors and prevents the use of mutable JavaBeans-style object creation. In SmartLib, the Builder pattern improves readability, supports validation during construction, and preserves the immutability of the final `Loan` object. The corresponding UML diagram is provided in `uml/builder-pattern.puml`.

The Decorator pattern is applied within the notification subsystem. `EmailNotificationService` represents the core notification service, while `LoggingDecorator`, `RetryDecorator`, and `RateLimitingDecorator` add extra behaviours dynamically. This approach is preferable to inheritance because notification features may need to be combined in different ways depending on runtime requirements. For example, a notification may require logging and retry support in one context, but rate limiting as well in another. The Decorator pattern therefore avoids subclass explosion and allows flexible composition. The corresponding UML diagram is provided in `uml/decorator-pattern.puml`.

The Observer pattern is used in the event subsystem through `LibraryEventBus`. This allows independent listeners, such as overdue fine handlers, member notification handlers, and audit log handlers, to respond to library events without being tightly coupled to the components that publish those events. This is particularly useful because a single event, such as a book return, may trigger several independent actions. The corresponding UML diagram is provided in `uml/observer-pattern.puml`.

Additional UML diagrams were also created to support the wider architectural discussion. These include the domain model, concurrency architecture, and sealed `LoanResult` hierarchy.

Together, the UML artefacts help clarify the relationships between classes, the use of composition, and the flow of control across the system.

Figure 1: Builder Pattern UML Diagram

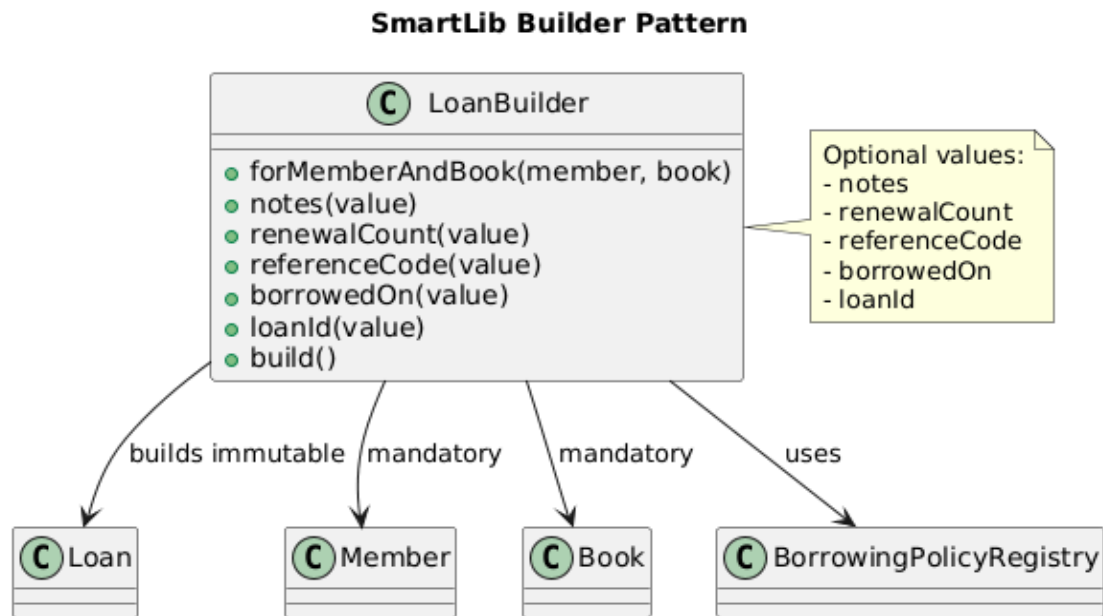


Figure 2: Decorator Pattern UML Diagram

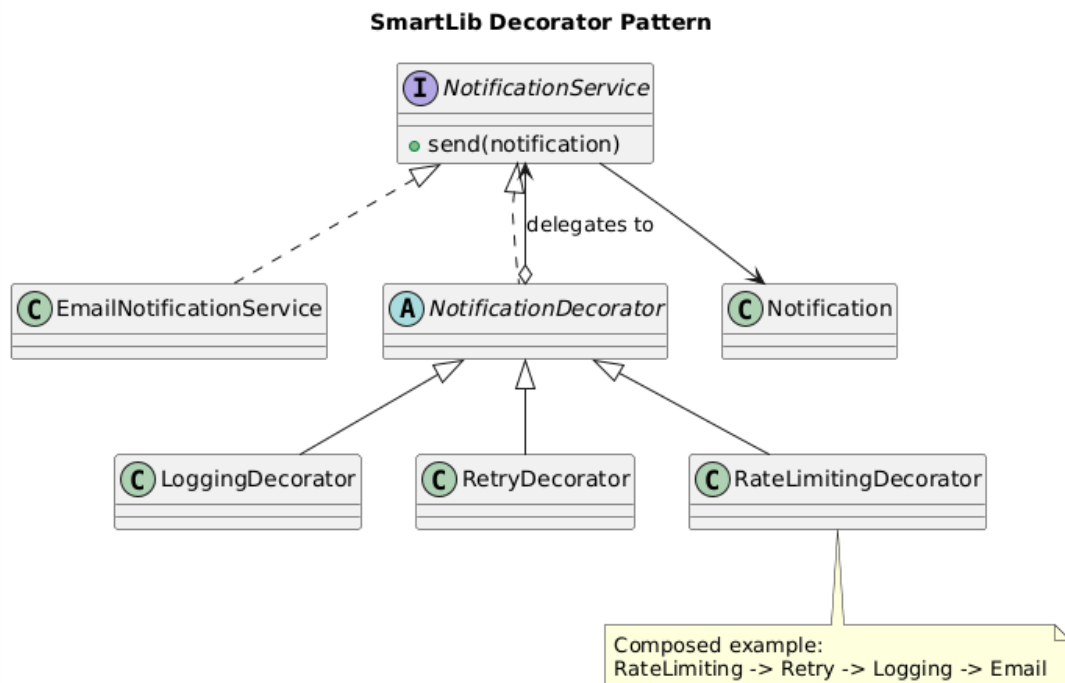
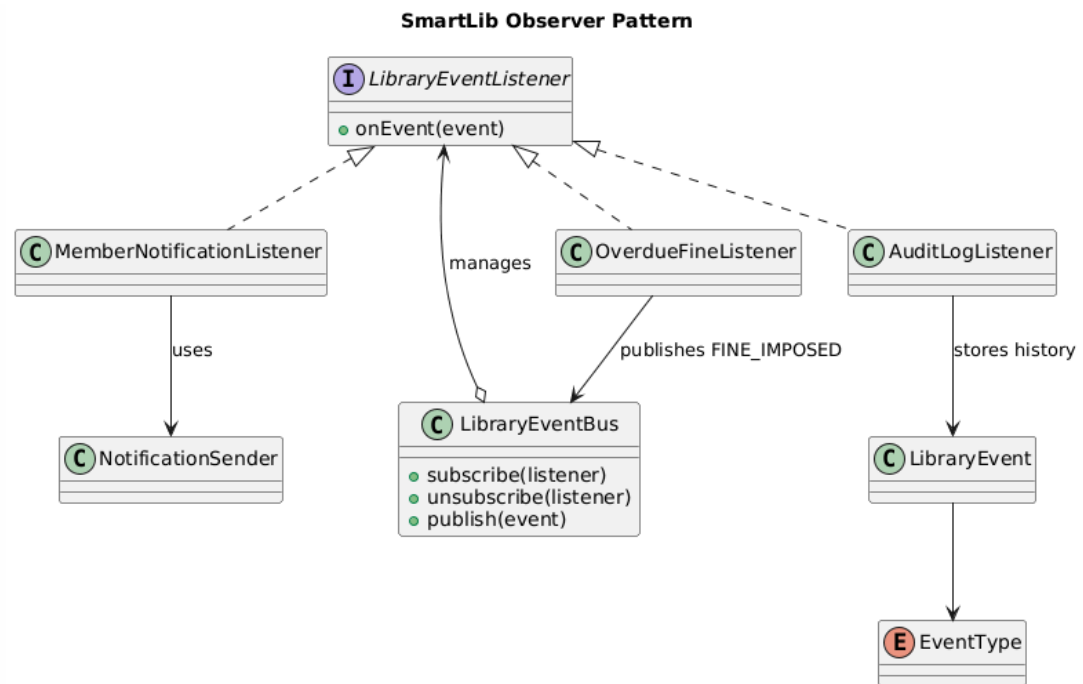


Figure 3: Observer Pattern UML Diagram



5.0 GENERICS AND TYPE SYSTEM DISCUSSION (QUESTION 3.2)

The generics component of SmartLib was developed to demonstrate how Java's type system can be used to create reusable, type-safe, and expressive APIs. Rather than using generics solely for code reuse, the implementation focuses on representing meaningful constraints and behavioural requirements within the type system itself. The most significant example is `Catalogue<T extends Borrowable & Comparable<T>>`, where catalogue items are required to support borrowing operations and provide a natural ordering. These bounds communicate the capabilities expected from catalogue entries while enabling compile-time validation of those requirements.

A key aspect of the implementation is the use of wildcard types following the PECS (Producer Extends, Consumer Super) principle. The method `addAllAvailable(Catalogue<? extends T>)` treats the supplied catalogue as a producer of values, whereas `updateMatching(Predicate<T>, Consumer<? super T>)` accepts a consumer capable of processing objects of type `T` or its supertypes. This approach improves API flexibility while maintaining strong type safety and reducing the risk of runtime type errors.

The project also investigates type erasure and its implications for runtime behaviour. Through `GenericUtils.safeCast` and a `TypeToken`-based reflection example, SmartLib demonstrates that generic type information is largely removed during compilation. As a result, parameterised types cannot usually be inspected directly at runtime unless additional metadata is preserved. Anonymous subclasses are used to retain type information where reflective analysis is required. Overall, this phase highlights both the advantages and challenges of Java generics. While generics significantly improve safety and expressiveness, concepts such as variance, wildcard bounds, and type erasure introduce additional complexity. By embedding these concepts within practical library-management examples, SmartLib provides a concrete demonstration of advanced generic programming techniques.

6. STREAM API AND FUNCTIONAL PROGRAMMING REFLECTIONS

The functional programming phase of SmartLib explored how Java Streams can be used to express data-processing logic in a more declarative and readable manner. Rather than relying on traditional iterative approaches, the project adopted stream pipelines to perform analytics tasks such as identifying the most frequently borrowed books, detecting overdue members, calculating average fines by membership category, generating sorted author lists, partitioning inventory by availability, and aggregating loans by month. These operations demonstrate how complex reporting requirements can be represented through concise sequences of transformations and reductions.

A key lesson from this phase was that streams are most beneficial when they clearly describe the flow of data through a series of well-defined operations. Collectors such as `groupingBy`, `counting`, `averagingDouble`, `joining`, and `partitioningBy` were chosen because they communicate intent directly and reduce the need for verbose control structures. As a result, the code becomes easier to understand and maintain.

The implementation of the custom `LoanSummaryCollector` provided further insight into the Stream API. While the collector uses a mutable accumulator internally to improve performance during aggregation, the final `LoanSummary` object is immutable. This demonstrates that functional programming in Java is not about eliminating mutation, but rather about restricting mutation to controlled and well-defined locations.

The project also incorporates method references, predicate composition, and function composition to improve code expressiveness and reuse. These features allow business rules and transformations to be represented as composable units of behaviour. Overall, this phase highlighted how functional programming techniques can improve clarity and maintainability when applied thoughtfully and in appropriate contexts.

7. CONCURRENCY DESIGN DECISIONS (QUESTION 5.3)

Concurrency was introduced into SmartLib with a strong emphasis on correctness, reliability, and maintainability rather than maximising raw performance. The objective was not simply to execute tasks in parallel, but to ensure that concurrent operations could be performed safely without compromising data integrity. This approach reflects the reality that correctness is often more important than throughput in business-critical applications.

The primary concurrency component is the `BookReturnProcessor`, which uses a fixed thread pool in combination with a `CompletableFuture` pipeline to process book returns asynchronously. The workflow is divided into a sequence of dependent stages, including validation, state updates, notification delivery, event publication, and monitoring. By modelling the process as a pipeline, the implementation remains easy to understand while still benefiting from asynchronous execution.

A key design decision was the separation of asynchronous orchestration from shared-state protection. Although `CompletableFuture` coordinates the execution of tasks, it does not inherently guarantee thread safety. For this reason, the critical section responsible for updating inventory, changing loan status, and persisting fines is protected by explicit locking. This ensures that state transitions remain consistent even when multiple return operations are processed concurrently.

To support lightweight monitoring, `AtomicInteger` was used to track processed and failed operations. Atomic variables provide thread-safe updates without requiring broader locking mechanisms, making them well suited to simple shared counters.

A second concurrency concern is addressed by the `InventoryManager`, which uses `ReentrantReadWriteLock` and a `Condition` object. This design allows multiple threads to read inventory information simultaneously while ensuring exclusive access during updates. The use of conditions also eliminates inefficient busy-waiting by allowing borrowing requests to block until stock becomes available. Overall, this phase demonstrated that effective concurrency design depends on careful control of shared state and a clear understanding of where synchronisation is truly required.

8. MODERN JAVA FEATURES DISCUSSION (QUESTION 6)

The final phase of SmartLib focused on the adoption of modern Java 17 features to improve code clarity, strengthen type safety, and reduce unnecessary complexity. Rather than introducing new language features purely for demonstration purposes, each feature was selected because it offered a practical advantage within the existing architecture.

One of the most significant additions was the sealed `LoanResult` hierarchy. Borrowing operations in the system can result in only a limited number of valid outcomes, including successful borrowing, insufficient stock, suspended membership, or excessive outstanding fines. A sealed hierarchy provides an explicit representation of these possibilities while preventing unauthorised extensions. Each result type was implemented as a record because these classes serve primarily as immutable data carriers and do not require complex behaviour.

Pattern matching was incorporated in two forms. Pattern matching for `instanceof` simplified object inspection by combining type checking and variable declaration into a single construct. Pattern-matching switch expressions were used within `LoanResultHandler` to process different result variants without the repetitive casting typically associated with traditional object-oriented approaches. Together, these features improved readability while reducing boilerplate code.

Switch expressions were also adopted for borrowing policy selection. Compared with conventional `if-else` chains or statement-based switches, switch expressions provide a more concise and expressive way to return values directly from control-flow logic. This reduces the likelihood of implementation errors and improves maintainability.

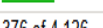
Text blocks were used when constructing JSON notification payloads. Because the resulting source code closely resembles the final JSON output, text blocks improve readability and reduce the need for extensive string concatenation or escaping. This makes structured text easier to maintain and modify.

Finally, the Java Platform Module System (JPMS) was introduced through `module-info.java`. Although SmartLib is not a large-scale application, the use of modules demonstrates how explicit dependency declarations and stronger encapsulation can be used to reinforce architectural boundaries. This phase highlighted how modern Java features can improve both developer productivity and software quality when applied thoughtfully.

9. TEST COVERAGE SUMMARY (INCLUDE JACOCO REPORT SCREENSHOT)

JaCoCo was used to measure the effectiveness of the automated test suite. The final coverage report indicates that the SmartLib system has been extensively tested across all major packages, including domain logic, design patterns, generics, functional programming, concurrency, and modern Java features

csp610-oop-smartlib

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 smartlib.domain		85%		62%	36	110	28	252	15	82	2	26
 smartlib.patterns.behavioural		89%		60%	11	29	12	96	5	21	1	6
 smartlib.functional		93%		59%	11	62	8	137	3	51	0	9
 smartlib.concurrent		95%		81%	6	50	5	143	3	42	0	10
 smartlib.patterns.structural		81%		60%	9	22	12	61	5	17	0	5
 smartlib.generics		92%		66%	3	22	4	52	1	19	0	5
 smartlib.patterns.creational		91%		71%	5	18	2	35	1	11	0	1
 smartlib.modern		99%		94%	1	30	1	82	0	19	0	8
Total	376 of 4,126	90%	50 of 158	68%	82	343	72	858	33	262	3	70

Understanding the Coverage Metrics

1. Instruction Coverage (90%)

Definition

Instruction coverage measures how many Java bytecode instructions were executed during testing.

From the report:

376 missed instructions out of 4,126 total

Coverage = 90%

Interpretation

This means approximately 90% of all executable code statements were exercised by the test suite.

A high instruction coverage indicates that most application logic has been executed and verified through automated tests.

Why it matters

Instruction coverage provides the broadest measure of testing completeness and is often considered the primary coverage metric.

2. Branch Coverage (68%)

Definition

Branch coverage measures whether all possible decision outcomes have been tested.

Examples include:

```
if (member.hasFine())
```

Both outcomes should be tested:

TRUE branch
FALSE branch

From the report:

50 missed branches out of 158 total
Coverage = 68%

Interpretation

The tests exercised approximately 68% of all logical decision paths.

The remaining uncovered branches are likely associated with:

- exceptional conditions
- defensive checks
- fallback logic
- retry mechanisms
- concurrency timing scenarios

Why it matters

Branch coverage is more demanding than instruction coverage because it verifies decision-making behavior rather than simply executing code.

3. Cyclomatic Complexity (Cxy)

Definition

Complexity estimates the number of independent execution paths through the code.

From the report:

343 total complexity points

82 not fully covered

Interpretation

Higher complexity indicates more decision points and possible execution paths.

The report shows that the majority of the application's logical paths have been tested.

Why it matters

Complexity helps identify areas that may require additional testing effort.

4. Line Coverage (92%)

Definition

Line coverage measures the percentage of source code lines executed during testing.

From the report:

72 missed lines out of 858 lines

Coverage $\approx$ 92%

Interpretation

Approximately 92% of all source code lines were executed by automated tests.

This is considered a very strong result for an academic software engineering project.

Why it matters

Line coverage provides a straightforward indication of how much of the application was exercised.

5. Method Coverage (87%)

Definition

Method coverage measures how many methods were called at least once during testing.

From the report:

33 missed methods out of 262 methods

Coverage $\approx$ 87%

Interpretation

Most methods in the system were executed by the test suite.

The remaining uncovered methods are likely utility methods, edge-case handlers, or supporting infrastructure code.

Why it matters

High method coverage demonstrates that functionality has been validated across a large portion of the codebase.

6. Class Coverage (96%)

Definition

Class coverage measures how many classes were exercised during testing.

From the report:

3 missed classes out of 70 classes

Coverage $\approx$ 96%

Interpretation

Almost every class in SmartLib was tested directly or indirectly.

This indicates excellent test penetration throughout the architecture.

Why it matters

High class coverage suggests that the overall system architecture has been thoroughly validated.

10. CONCLUSION AND LESSONS LEARNED

The SmartLib project demonstrates how a relatively compact software system can be used to explore and apply a broad range of advanced software engineering concepts. Although the application operates within a modest library management domain, it provides a suitable environment for examining object-oriented design, architectural refactoring, design patterns, generic programming, functional programming, concurrency, and modern Java language features. By introducing these concepts incrementally, it was possible to evaluate their individual contributions while maintaining a coherent and stable architecture throughout the development process.

Several important lessons emerged during implementation. First, separating business logic from infrastructure concerns significantly improves maintainability, testability, and long-term flexibility. Second, design patterns provide the greatest value when they address genuine design pressures rather than being applied solely for academic purposes. Third, advanced language features such as generics and streams become easier to understand and justify when they solve concrete domain problems. Fourth, concurrency requires careful management of shared state regardless of the abstraction level provided by modern APIs. Finally, modern Java features including records, sealed hierarchies, pattern matching, and switch expressions can improve clarity and reduce boilerplate when used appropriately.

Overall, the project reinforces the principle that effective software design is not measured by the number of technologies employed, but by the extent to which each design decision contributes to clarity, maintainability, and correctness. SmartLib therefore serves as both a functional application and a practical demonstration of modern object-oriented software engineering principles.

11. REFERENCES

- I. Bauer, L., Appel, A. W., & Felten, E. W. (2003). Mechanisms for secure modular programming in Java. **Software: Practice and Experience*, 33*(5), 461-480.
<https://doi.org/10.1002/spe.516>
- II. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). **Design patterns: Elements of reusable object-oriented software**. Addison-Wesley.
- III. Liskov, B., & Wing, J. M. (1994). A behavioral notion of subtyping. **ACM Transactions on Programming Languages and Systems*, 16*(6), 1811-1841.
<https://doi.org/10.1145/197320.197383>
- IV. Rosales, E., Basso, M., Rosa, A., & Binder, W. (2023). Large-scale characterization of Java streams. **Software: Practice and Experience*, 53*(9), 1763-1792.
<https://doi.org/10.1002/spe.3213>
- V. Strniša, R., Sewell, P., & Parkinson, M. (2007). The Java module system: Core design and semantic definition. In **Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications** (pp. 499-514).
<https://www.microsoft.com/en-us/research/publication/the-java-module-system-core-design-and-semantic-definition/>
- VI. Tate, R., Leung, A., & Lerner, S. (2011). Taming wildcards in Java's type system. In **Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation** (pp. 611-622).
<https://doi.org/10.1145/1993498.1993570>
- VII. Zhang, H., Xie, Y., Yue, L., & Qi, Y. (2024). Automatic refactoring approach for asynchronous mechanisms with CompletableFuture. **Applied Sciences*, 14*(19), Article 8866. <https://doi.org/10.3390/app14198866>